QueryLocal

Ask your database in plain English, privately and offline

A complete guide to the project: what it does, the ideas behind it, how every part works, how it was built stage by stage, the benchmark results, the problems we hit along the way, and every terminal command you need.

Project folder: E:\Projects\QueryLocal · GitHub repository: QueryLocal · Guide version: 16 October 2026

Contents

1. What QueryLocal is **4**

1.1 Why this project is interesting . . . 4

1.2 Headline results . . . 4

2. Key ideas, explained from zero **5**

3. Tools and requirements **6**

3.1 The machine it was built on . . . 6

3.2 Software and libraries . . . 6

4. Project structure **7**

4.1 What each file does . . . 7

4.2 The sample database . . . 8

5. How it works **9**

5.1 Step 1: reading the schema . . . 9

5.2 Step 2: interpreting the question . . . 9

5.3 Step 3: the safety check . . . 10

5.4 Step 4: running the query read-only . . . 10

5.5 Step 5: self-correction . . . 10

5.6 The two backends . . . 10

5.7 The web app . . . 10

6. Safety and privacy design **11**

7. How it was built, stage by stage **12**

8. The benchmark **14**

8.1 How it works . . . 14

8.2 Results . . . 14

8.3 What the results show . . . 15

8.4 Where the small local model goes wrong . . . 15

8.5 Limits of these numbers . . . 15

8.6 The prompt improvement (Stage 7) . . . 16

9. Problems we hit and how we fixed them **17**

10. Set it up from scratch **18**

11. Command reference **19**

11.1 Python and the virtual environment . . . 19

11.2 Packages . . . 19

11.3 Git and GitHub . . . 19

11.4 Running QueryLocal . . . 20

11.5 Ollama . . . 20

11.6 Benchmark, comparison and charts . . . 21

11.7 Handy checks . . . 21

11.8 Keyboard shortcuts in VS Code . . . 22

12. Ideas for next steps **23**

Resume summary . . . 23

1. What QueryLocal is

QueryLocal is a **text-to-SQL** application. You type a question about your data in normal English, for example *"Who are the top 3 customers by total order amount?"*. QueryLocal writes the SQL query that answers it, checks that the query is safe, runs it on the database and shows you the result as a table.

It can use two kinds of AI model, and you switch between them with one click:

- **A cloud model:** Google Gemini (`gemini-3.5-flash-lite`), reached over the internet through Google's API. It is the most accurate option, but your question and your table definitions are sent to Google.
- **A local model:** a model that runs on your own PC through Ollama (`qwen2.5-coder:3b` by default). Nothing leaves your computer and it works with Wi-Fi turned off, but it is less accurate.

1.1 Why this project is interesting

- Most people who need data cannot write SQL. Text-to-SQL lets them ask questions directly.
- Companies often cannot send their data to a cloud AI. A local model solves that privacy problem, so measuring how much accuracy you give up is a real, useful question.
- The project does not just generate SQL. It is built to be **safe** (it can never change or delete data), **self-correcting** (it fixes its own SQL errors) and **honest** (it states its assumptions, asks when a question is unclear, and says when the data cannot answer it).
- It is **measured**: both kinds of model were tested on Spider, the standard public text-to-SQL benchmark.

1.2 Headline results

Model	Where it runs	Accuracy (50 questions)	Average time
`gemini-3.5-flash-lite`	Cloud (Google)	88%	9.7 s
`gemma4:12b`	Local (Ollama)	74%	71.5 s
`qwen2.5-coder:3b`	Local (Ollama)	70%	10.9 s
`qwen3.5:4b`	Local (Ollama)	60%	23.2 s
`llama3.2:3b`	Local (Ollama)	58%	10.0 s

The short version: the cloud model is clearly the most accurate. The best everyday local model is the small code-specialised `qwen2.5-coder:3b`, which is about as fast as Gemini on a CPU-only laptop. A model four times bigger (`gemma4:12b`) gained only 2 extra correct answers and was about 7 times slower. Section 8 explains how these numbers were measured and what they do and do not prove.

2. Key ideas, explained from zero

You do not need to know any of these terms before reading this guide. Each one is used later.

Term	What it means here
Database	An organised store of data in tables (like spreadsheets with strict columns). QueryLocal uses **SQLite**, where a whole database is one file, for example `data/shop.db`.
Table, row, column	A table such as `customers` has columns (`name`, `city`) and one row per customer.
SQL	Structured Query Language, the language used to ask a database for data. Example: `SELECT city, COUNT(*) FROM customers GROUP BY city`.
Schema	The description of the tables and their columns, written as `CREATE TABLE` statements. The model needs it to know what exists.
Query	One SQL request. QueryLocal only ever runs read-only `SELECT` queries.
LLM	Large Language Model, an AI trained on huge amounts of text that can write text and code. Gemini, Qwen, Llama and Gemma are all LLMs.
Prompt	The text we send to the model: the schema, our rules and the user's question.
API and API key	An API lets a program talk to an online service. The API key is a secret password that identifies your account. It lives only in the `.env` file.
Cloud vs local model	A cloud model runs on someone else's servers. A local model runs on your own computer.
Ollama	A free app that downloads open models and runs them on your PC, with a Python package to call them.
Parameters (3b, 12b)	The model's size in billions of numbers. Bigger usually means smarter but slower and more memory hungry.
JSON	A simple text format for structured data, like `{"status": "ok", "sql": "..."}`. We ask the model to reply in JSON so the program can read its answer reliably.
Virtual environment (venv)	A private folder (`.venv`) holding this project's Python and packages, so they do not clash with other projects.
pip	Python's package installer. `requirements.txt` lists every package the project needs.
Git and GitHub	Git records versions of your code (commits). GitHub stores them online (push).
Streamlit	A Python library that turns a script into a web page with inputs, buttons and tables.
Benchmark	A fixed set of test questions with known correct answers, used to score models fairly.
Spider	The standard public text-to-SQL benchmark from Yale: about 1,000 dev questions over 200 databases.
Execution accuracy	A question is correct if our SQL returns exactly the same rows as the official correct SQL. The SQL text itself is never compared.
Rate limit / quota	The maximum number of requests a free API key may send per minute or per day.

3. Tools and requirements

3.1 The machine it was built on

- Windows with PowerShell inside VS Code.
- 16 GB of RAM and **no dedicated graphics card** (only a 128 MB Intel chip). Every local model therefore ran 100% on the CPU, which is why local timings are slower than they would be on a gaming PC.
- Python 3.12 from python.org. (An MSYS2 Python already on the PC caused problems; see section 9.)

3.2 Software and libraries

Tool	Used for
Python 3.12	The programming language for everything.
`google-genai`	Google's official SDK for calling Gemini.
`python-dotenv`	Loads `GEMINI_API_KEY` from the `.env` file.
`streamlit`	The web interface.
`pandas`	Shows results as tables in Streamlit (installed with Streamlit).
Ollama app + `ollama` package	Runs local models and lets Python call them.
`matplotlib`	Draws the benchmark charts.
`sqlite3`	Built into Python, used to read the databases.
Git + GitHub	Version control and the online repository `QueryLocal`.
Spider dataset	The benchmark questions and their databases, stored in `data/spider/`.

4. Project structure

```
E:\Projects\QueryLocal
├── .env                      your Gemini key (secret, never committed)
├── .gitignore                tells Git what to leave out
├── .venv\                    virtual environment (never committed)
├── requirements.txt          list of Python packages
├── README.md                 project page shown on GitHub
├── docs\
│   └── demo.png              screenshot used in the README
├── data\
│   ├── shop.db               sample database (generated, not committed)
│   └── spider\               Spider benchmark data (about 1 GB, not committed)
│       ├── dev.json
│       └── database\<db_name>\<db_name>.sqlite
├── src\
│   ├── text_to_sql.py        the engine: prompts, safety, retries, both backends
│   └── app.py                the Streamlit web interface
├── scripts\
│   ├── hello_gemini.py       first "is my key working?" test
│   ├── create_sample_db.py   builds data\shop.db
│   ├── benchmark.py          runs the Spider benchmark
│   ├── compare.py            compares two result files question by question
│   └── plot_results.py       draws the accuracy charts
└── results\
    ├── <backend>_<model>_<n>.csv   one file per benchmark run
    ├── comparison.md               output of compare.py
    ├── accuracy.png                output of plot_results.py
    └── accuracy_vs_speed.png
```

4.1 What each file does

File	Purpose
`src/text_to_sql.py`	The heart of the project. Reads the schema, builds prompts, calls Gemini or Ollama, validates SQL, runs it read-only, retries on errors, handles clarifying questions. Can also be run on its own as a terminal chat.
`src/app.py`	The web page. Sidebar with the schema and a Gemini/Ollama switch, a question box, and output showing SQL, assumption, result table and timing.
`scripts/create_sample_db.py`	Creates `data/shop.db` with 10 customers in 5 Indian cities and 200 random orders from the last 180 days.
`scripts/hello_gemini.py`	Sends one line to Gemini to prove the key and model name work.
`scripts/benchmark.py`	Picks N random Spider questions (seed 42), runs them through the engine and saves one CSV row per question. Resumable.
`scripts/compare.py`	Reads two result CSVs and writes `results/comparison.md` with a metrics table and the questions only one model got right.
`scripts/plot_results.py`	Reads every full-length result CSV and draws the two charts. Skips short test runs.
`.gitignore`	Keeps `.env`, `.venv`, `data/*.db` and `data/spider/` out of GitHub.

4.2 The sample database

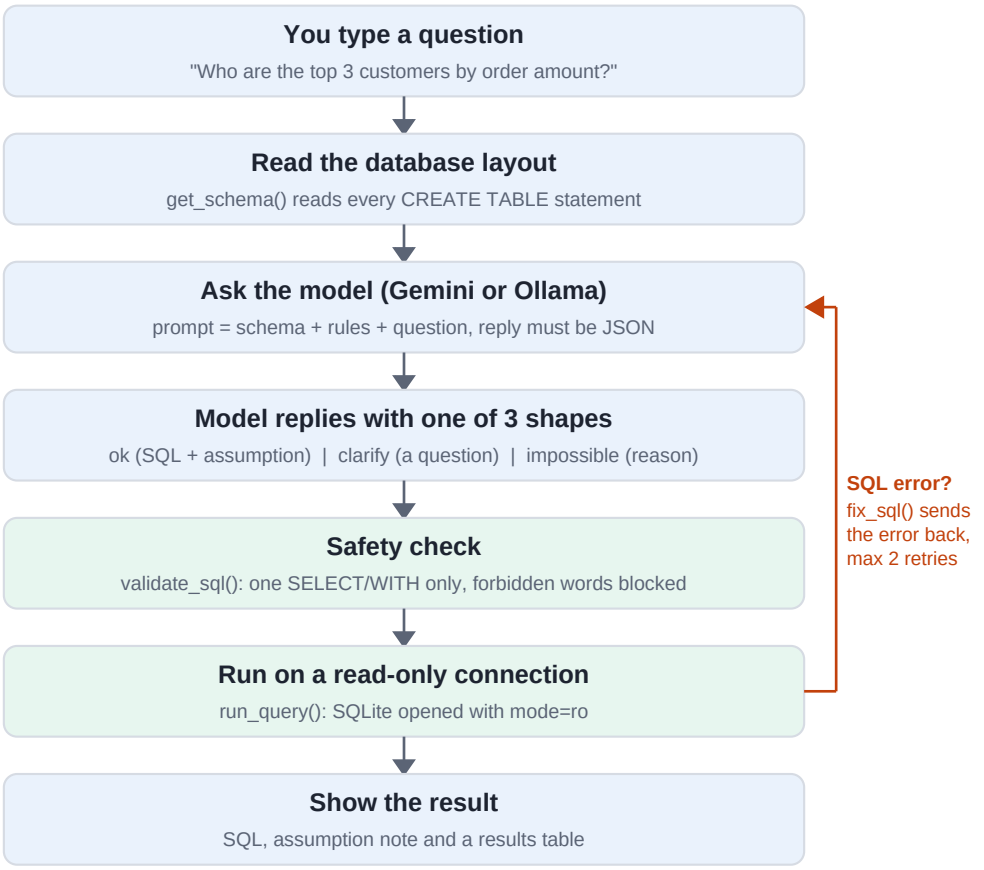
```
CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY,
    name        TEXT NOT NULL,
    city        TEXT,
    signup_date DATE
);

CREATE TABLE orders (
    order_id    INTEGER PRIMARY KEY,
    customer_id INTEGER REFERENCES customers(customer_id),
    order_date  DATE,
    amount      REAL
);
```

`orders.customer_id` points to `customers.customer_id`, so each order belongs to one customer. Questions such as "total spent per city" need a **JOIN** between the two tables.

5. How it works

Every question, from the web app, the terminal or the benchmark, goes through the same function: `answer()` in `src/text_to_sql.py`. This diagram shows the path a question takes.



5.1 Step 1: reading the schema

`get_schema(db_path)` asks SQLite for the `CREATE TABLE` statement of every table and joins them into one block of text. This is all the model knows about your data, which is why it never needs to see the data itself.

5.2 Step 2: interpreting the question

`interpret()` sends the model a prompt containing the schema, today's date, the question and these instructions. The model must reply with JSON in exactly one of three shapes:

```
{"status": "ok", "sql": "<one SQLite SELECT query>", "assumption": "<what you assumed, or empty>"}
{"status": "clarify", "question": "<one short question for the user>"}
{"status": "impossible", "reason": "<why this database cannot answer it>"}
```

- **ok**: the model wrote SQL. If the question was a little vague, it picks a sensible default and says so. For "top customers" it assumes "highest total order amount"; for "recent" it assumes "last 30 days". The app shows this as a blue *Assumption* note.
- **clarify**: used only when the question could mean clearly different things and no sensible default exists. The app shows the question and waits for your answer, at most 2 rounds.

- **impossible**: the needed tables or columns do not exist. For "total sales per product category" it answers at once that there is no category data, instead of wasting retries.

If the model ignores the format and returns plain text, the code treats that text as SQL, so a badly behaved model still gets a chance.

5.3 Step 3: the safety check

`validate_sql()` rejects anything that is not exactly one read-only query:

- empty queries, and more than one statement (any `;` in the middle);
- anything that does not start with `SELECT` or `WITH`;
- any of the blocked words: `DROP DELETE UPDATE INSERT ALTER CREATE REPLACE ATTACH DETACH PRAGMA VACUUM TRUNCATE`.

5.4 Step 4: running the query read-only

`run_query()` opens the database with `file:...?mode=ro`, SQLite's read-only mode. Even if a harmful query slipped past the safety check, SQLite itself would refuse to write. This is the second layer of protection. It also tolerates oddly encoded text in some Spider databases.

5.5 Step 5: self-correction

If validation or execution fails (for example `no such column: T2.owner_id`), `fix_sql()` sends the model the question, the failed SQL and the exact error message, and asks for a corrected query. This happens at most `MAX_RETRIES = 2` times, so a question gets 3 attempts in total.

5.6 The two backends

	Gemini (cloud)	Ollama (local)
Function	`ask_gemini()`	`ask_ollama()`
Model setting	`MODEL = "gemini-3.5-flash-lite"`	`OLLAMA_MODEL = "qwen2.5-coder:3b"`
JSON mode	`response_mime_type="application/json"`	`format="json"`
Special settings	Retries with waiting when Google is busy (503) or rate limits (429). Stops at once if the **daily** quota is used up, because retrying would only waste requests.	`think=False` (no slow reasoning phase), `temperature 0` (same answer every time), `num_predict 512` (caps reply length), 120-second timeout so it can never hang forever.

`ask_model()` picks the backend and prints how long each call took, for example `[ollama call took 11.2s]`. In the terminal version the backend comes from the `BACKEND` environment variable; in the web app it comes from the sidebar switch.

5.7 The web app

`streamlit run src/app.py` opens `http://localhost:8501`. The sidebar shows the schema and a Gemini/Ollama radio button. Pending clarifying questions are kept in `st.session_state` so the page remembers them between clicks. Results show the generated SQL, the assumption note, a table and the time taken.

6. Safety and privacy design

Risk	How QueryLocal handles it
The model writes `DELETE` or `DROP`	Blocked by `validate_sql()`, and the read-only connection would refuse it anyway.
Several statements chained with `;`	Rejected: only one statement is allowed.
The model invents a column	The SQL error is sent back and the model gets 2 chances to fix it.
A vague question gets a silent guess	The model must state its assumption, or ask one clarifying question.
An impossible question	The model says so immediately instead of making something up.
Leaking the API key	The key lives only in `.env`, which `.gitignore` keeps off GitHub. Never paste it into the README or code.
Sending data to the cloud	Only the schema and question go to Gemini, never the rows. With Ollama, nothing leaves the PC at all.
The app hanging forever	Ollama calls time out after 120 seconds; Gemini retries are capped.

Test that proves the data is safe: ask "Delete all orders" in the app, then run this. It must still print `(200,)`.

```
python -c "import sqlite3; print(sqlite3.connect('data/shop.db').execute('select count(*) from orders').fetchone())"
```

7. How it was built, stage by stage

The project was built in small stages. Each stage ended with a working program and a Git commit.

Stage 0: Setup

Installed Python 3.12, created the virtual environment, installed `google-genai` and `python-dotenv`, created the `.env` key file, the folders and `.gitignore`, and proved the key works with `scripts/hello_gemini.py`.

Commit message: *Set up project structure and test Gemini API*

Stage 1: Basic text-to-SQL

Built the sample shop database and a first engine: schema in, SQL out, run it, print the rows. Tested with: customers per city, top 3 customers by total order amount, revenue in the last 30 days.

Commit message: *Add script to create sample shop database / Add basic Text-to-SQL engine with Gemini*

Stage 2: Safety and self-correction

Added `validate_sql()`, the read-only connection and the 2-retry `fix_sql()` loop.

Commit message: *Add SQL safety validation and self-correction loop*

Stage 3: Web interface

Built `src/app.py` with Streamlit.

Commit message: *Add Streamlit web interface*

Stage 2b: Vague and impossible questions

Switched the model's reply to JSON with ok / clarify / impossible, added assumptions and up to 2 clarification rounds, in both the terminal and the web app. Done before the local models so it only had to be built once.

Commit message: *Handle ambiguous and unanswerable questions with clarification*

Stage 4: Local models with Ollama

Added `ask_ollama()` and the sidebar switch. `qwen3.5:4b` was too slow on a CPU (it reasons before answering), so the default became `qwen2.5-coder:3b` at about 11 s per question.

Commit message: *Add local Ollama backend with model switch in the UI*

Stage 5: Benchmark

Downloaded Spider, wrote `benchmark.py` and `compare.py`, handled Gemini overload and quota errors, and switched to `gemini-3.5-flash-lite` because the bigger model allowed only 20 free requests per day. Result: Gemini 44/50, local 35/50.

Commit message: *Add Spider benchmark script / Add benchmark results and comparison script*

Stage 6: README and demo

Wrote the README with real numbers and a failure analysis, and added the demo screenshot `docs/demo.png`.

Commit message: *Add README with benchmark results and demo screenshot*

Stage 7: More models, charts and a prompt improvement

Added a `--model` option to the benchmark and tested `llama3.2:3b`, `qwen3.5:4b` and `gemma4:12b`. Added `plot_results.py` for the charts. Then started a prompt improvement: showing the model 3 example rows from each table (`--sample-rows 3`). *At the time of writing, this last run was still in progress.*

Commit message: *(not yet committed)*

8. The benchmark

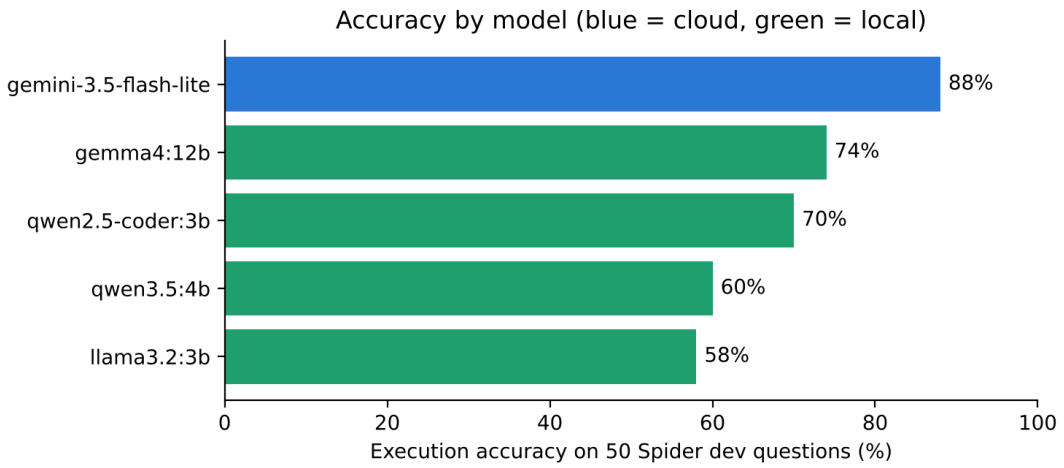
8.1 How it works

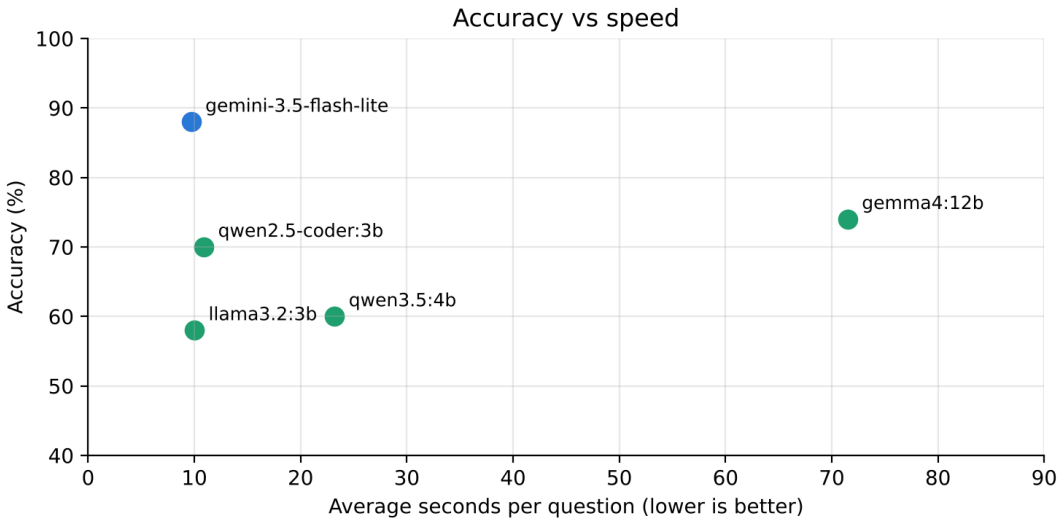
1. **Questions.** Spider's `dev.json` holds about 1,000 questions written by people, each with the correct SQL and the name of its database.
2. **Sampling.** `random.Random(42).sample(examples, n)` picks the same `n` questions every time. Every model got exactly the same 50 questions. Different `--n` values give different sets, so always compare runs with the same `--n`.
3. **Answering.** Each question goes through the normal `answer()` function on its own database, with the same safety check and retries.
4. **Scoring.** Both the model's SQL and the correct SQL are run, and the rows are compared with `Counter(map(tuple, rows))`. Row order is ignored, duplicates count.
5. **Saving.** One CSV row per question (`idx, db_id, question, gold_sql, pred_sql, status, correct, seconds, error`), written immediately, so a stopped run resumes where it left off.

Anything other than matching rows counts as wrong: an SQL error after retries, a clarifying question (the benchmark cannot answer it), "impossible", or different rows.

8.2 Results

Model	Correct	Accuracy	Avg time	Notes
`gemini-3.5-flash-lite`	44/50	88%	9.7 s	0 errors, 6 wrong results; median 6.5 s
`gemma4:12b`	37/50	74%	71.5 s	2 errors
`qwen2.5-coder:3b`	35/50	70%	10.9 s	3 errors, 12 wrong results; median 8.3 s
`qwen3.5:4b`	30/50	60%	23.2 s	1 clarify
`llama3.2:3b`	29/50	58%	10.0 s	3 errors, 5 clarify





8.3 What the results show

- **Cloud beats local on accuracy.** Gemini got 10 questions right that `qwen2.5-coder:3b` missed, and the reverse happened only once. A sign test on those 10 vs 1 gives p of about 0.01, so the gap is unlikely to be luck.
- **A small specialist beats a bigger generalist.** The code-trained `qwen2.5-coder:3b` (70%) beat the general `qwen3.5:4b` (60%) and was about twice as fast.
- **Bigger is not automatically worth it on a CPU.** `gemma4:12b` is 4 times bigger but gained only 2 questions (74% vs 70%, too close to call with 50 questions) at about 7 times the time.
- **Some misses are caution, not errors.** `llama3.2:3b` asked 5 clarifying questions, which the benchmark counts as wrong.

8.4 Where the small local model goes wrong

From the 10 questions only Gemini got right:

1. **Wrong table or column** (about half). All 3 SQL errors were invented columns such as `T2.owner_id`. Others used a plausible but wrong table.
2. **Missing joins.** For "the airport with the least flights" it counted airports, never joining the flights table.
3. **Negation.** For "students who do not have a cat" it used `WHERE pettype != 'cat'`, which still returns students who own a cat and another pet. It needs `NOT IN` or `EXCEPT`.

At least one "wrong" answer was logically right but left out a duplicated column that the official answer repeats, so the true gap is probably a little smaller than the raw numbers say.

8.5 Limits of these numbers

- 50 questions is a small sample. Treat each percentage as roughly plus or minus 9 points.
- The check is strict: different column order, an extra column or tiny rounding differences count as wrong.
- Gemini timings include network time and free-tier waits. Local timings depend on this CPU-only PC.
- The Gemini model was chosen partly because larger models had a free limit of 20 requests per day.

8.6 The prompt improvement (Stage 7)

Because wrong table and column choices were the biggest weakness, the next experiment shows the model 3 real example rows from every table, below each `CREATE TABLE`. The new `get_schema(db_path, sample_rows)` does this when `sample_rows` is above 0, and the benchmark's `--sample-rows 3` option saves the result as a separate file (`ollama_qwen2.5-coder_3b-rows3_50.csv`), so the charts show before and after side by side. Compare it with the 70% baseline when it finishes.

9. Problems we hit and how we fixed them

Every one of these happened during the build. They are the most likely problems for anyone setting it up again.

Symptom	Cause	Fix
Activate.ps1 "module could not be loaded"	PowerShell needs `.\` in front of a script path.	Use `.\.venv\Scripts\Activate.ps1`.
.venv has no Scripts folder (it has bin)	The `python` on PATH was MSYS2's, which builds Linux-style venvs.	Install Python 3.12 from python.org, delete `.venv`, recreate with `py -3.12 -m venv .venv`.
"running scripts is disabled"	Windows blocks PowerShell scripts by default.	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, then type `Y`.
ModuleNotFoundError: dotenv (or ollama)	Running with VS Code's Run button uses the system Python; or the package was not installed in the venv.	Run from the terminal with (.venv) showing. Install missing packages with `pip install ...`.
can't open file src/text_to_sql.py	The file did not exist yet or was saved elsewhere.	Check with `dir src` and create it inside `src`.
First Gemini question is slow	Connection setup and model start-up on the first call.	Normal; later questions are quick. (Deeper investigation deferred.)
Ollama spins forever in Streamlit	`qwen3.5:4b` reasons before answering, and on a CPU with a long prompt that takes minutes.	Added `think=False`, `num_predict 512` and a 120 s timeout, and switched to `qwen2.5-coder:3b`.
Gemini 503 UNAVAILABLE	Google's servers were overloaded.	Automatic retry with growing waits (5, 10, 20, 40, 60 s).
Gemini 429 RESOURCE_EXHAUSTED	Free quota: `gemini-3.8-flash` allowed only 20 requests per day.	Stop at once on daily-quota errors, and switch to `gemini-3.5-flash-lite`, which has its own larger quota.
Live API / Veo models in the rate-limit list	They are voice/video models with a different API.	Use only text Flash, Flash-Lite or Gemma models.
A 100% bar in the chart	An old 2-question test file was still in `results/`.	Delete test files; `plot_results.py` now skips shorter runs automatically.
Screenshot pasted as text in README	VS Code saved the image with a random name and added a link.	Move it to `docs/demo.png` and point the image link there.

10. Set it up from scratch

Follow these steps on a new Windows PC. Run every command in a VS Code terminal (PowerShell) inside the project folder.

Step 1: Install the tools

- Python 3.12 from python.org (tick **Add python.exe to PATH**).
- Git from git-scm.com.
- Ollama from ollama.com/download (only needed for local models).
- VS Code.

Step 2: Get the code and create the environment

```
git clone https://github.com/<your-username>/QueryLocal.git
cd QueryLocal
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Step 3: Add your Gemini key

Get a free key at aistudio.google.com (Get API key, then Create API key). Create a file called `.env` in the project folder with one line, no quotes:

```
GEMINI_API_KEY=your_key_here
```

Step 4: Create the sample database and try it

```
python scripts/create_sample_db.py
python scripts/hello_gemini.py
streamlit run src/app.py
```

Step 5: Add a local model

```
ollama pull qwen2.5-coder:3b
ollama run qwen2.5-coder:3b "Write a SQL query that counts rows in a table called orders"
```

Type `/bye` to leave the Ollama chat, then pick **Ollama (local)** in the app's sidebar.

Step 6 (optional): Reproduce the benchmark

Download Spider from its official page (search "Spider Yale text-to-SQL") and unzip it so that `data\spider\dev.json` and `data\spider\database\` exist. Check that `data/spider/` is in `.gitignore`, then run the commands in section 11.6.

11. Command reference

All commands are for PowerShell in VS Code, run from `E:\Projects\QueryLocal`. Most need `(.venv)` showing at the start of the prompt.

11.1 Python and the virtual environment

Command	What it does
`py -3.12 -m venv .venv`	Create the virtual environment (once).
`.\.venv\Scripts\Activate.ps1`	Activate it. Do this **every time** you open a new terminal.
`deactivate`	Leave the virtual environment.
`python --version`	Check which Python is active (should be 3.12.x).
`where.exe python`	Show every python.exe on PATH (found the MSYS2 problem).
`Remove-Item -Recurse -Force .venv`	Delete a broken venv so it can be recreated.
`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`	Allow PowerShell to run the activate script.

11.2 Packages

Command	What it does
`pip install -r requirements.txt`	Install everything the project needs.
`pip install google-genai python-dotenv`	Gemini SDK and .env loader (Stage 0).
`pip install streamlit`	Web interface (Stage 3).
`pip install ollama`	Python package for local models (Stage 4).
`pip install matplotlib`	Charts (Stage 7).
`pip freeze > requirements.txt`	Save the current package list. Run after every install.
`pip list`	Show installed packages.

11.3 Git and GitHub

Command	What it does
`git status`	See what changed. Check that .env, .venv and data/spider are **not** listed.
`git add .`	Stage all changes for the next commit.
`git commit -m "message"`	Save a version with a description.
`git push`	Upload commits to GitHub.
`git log --oneline`	List previous commits.
`git clone https://github.com/<user>/QueryLocal.git`	Download the repository on a new PC.

The everyday save routine used after every stage:

```
git status
git add .
git commit -m "Describe what you changed"
git push
```

11.4 Running QueryLocal

Command	What it does
`python scripts/create_sample_db.py`	Build data/shop.db.
`python scripts/hello_gemini.py`	Check the API key and model name.
`python src/text_to_sql.py`	Terminal chat using Gemini. Type quit to exit.
`$env:BACKEND="ollama" python src/text_to_sql.py`	Terminal chat using the local model (this terminal only).
`streamlit run src/app.py`	Start the web app at http://localhost:8501.
`Ctrl+C`	Stop the app, a chat or a benchmark run.

11.5 Ollama

Command	What it does
`ollama --version`	Check Ollama is installed.
`ollama pull qwen2.5-coder:3b`	Download a model (also llama3.2:3b, qwen3.5:4b, gemma4:12b).
`ollama list`	Show downloaded models and their exact names.
`ollama run qwen2.5-coder:3b "..."`	Ask a model directly. /bye exits.
`ollama ps`	Show loaded models and whether they use CPU or GPU.
`ollama stop qwen3.5:4b`	Unload a stuck or idle model to free memory.

11.6 Benchmark, comparison and charts

Command	What it does
`python scripts/benchmark.py --backend ollama --n 50`	Benchmark the default local model on 50 questions.
`python scripts/benchmark.py --backend ollama --model gemma4:12b --n 50`	Benchmark any local model.
`python scripts/benchmark.py --backend gemini --n 50 --delay 8`	Benchmark Gemini, pausing 8 s between questions to stay under the per-minute limit.
`... --n 2`	Quick test run. Delete its CSV afterwards.
`... --retry-errors`	Re-run only rows that ended in an error (useful for Gemini outages, not for real model mistakes).
`... --sample-rows 3`	Show 3 example rows per table in the prompt (Stage 7 experiment).
`python scripts/compare.py results/A.csv results/B.csv`	Compare two runs; writes results/comparison.md.
`python scripts/plot_results.py`	Draw results/accuracy.png and results/accuracy_vs_speed.png.
`dir results`	List result files.
`Remove-Item results\*_2.csv`	Delete 2-question test files.

11.7 Handy checks

Command	What it does
`dir src / dir data\spider`	Check files are where they should be.
`python -c "import sqlite3; print(sqlite3.connect('data/shop.db').execute('select count(*) from orders').fetchone())"`	Prove no data was deleted (prints (200,)).
`python -c "from dotenv import load_dotenv; from google import genai; load_dotenv(); [print(m.name) for m in genai.Client().models.list()]"`	List every Gemini model your key can use.
`python -c "import sys; sys.path.insert(0,'src'); import text_to_sql as t; print(t.client.models.generate_content(model=t.MODEL, contents='hi').text)"`	Send one test request to see the exact quota error, if any.

11.8 Keyboard shortcuts in VS Code

Shortcut	What it does
Ctrl+`	Open or close the terminal.
Ctrl+Shift+P, then Python: Select Interpreter	Make the Run button use the .venv Python.
Ctrl+Shift+V	Preview a Markdown file such as the README.
Ctrl+F	Find text in the open file.
Win+Alt+R	Start or stop a screen recording (Xbox Game Bar) for a demo video.

12. Ideas for next steps

- Finish the example-rows experiment and add its before/after chart to the README.
- Fix the slow first Gemini answer (deferred during the build).
- Add few-shot examples to the prompt for negation and set questions (`NOT IN`, `EXCEPT`).
- Benchmark on more than 50 questions to tighten the error bars.
- Support PostgreSQL or MySQL, and let users upload their own SQLite file in the app.
- Show an answer in plain English below the table, not just rows.

Resume summary

Built QueryLocal, a text-to-SQL app in Python and Streamlit that runs on Google Gemini or local Ollama models, with read-only safety checks, automatic SQL self-correction and handling for vague and unanswerable questions. Benchmarked five models on the Spider dataset: 88% accuracy with Gemini against 70% for a 3B local code model running on a CPU, and showed that a 12B local model added little accuracy at about seven times the latency.